You call in the guy, and he immediately sees the problem for what it is: the 0-1 knapsack problem. The bad news is that it is a NP-complete[7] problem, and this means it is hard to solve, as there is no “quick” (as in polynomial with respect to the number of items) algorithm that computes the optimal solution.

There is, however, also good news: there exist a pseud-polynomial solution, a solution using *dynamic programming*[8] that takes time proportional to the max capacity of the knapsack, and luckily the capacity of the crate is limited: it takes a number of steps equal to the number of possible values for the capacity multiplied by the number of items, so assuming the smallest unit is 1kg, it only takes 1000 * 62 steps. Wow, that’s much better than 2^{62}! And, in fact, once you rewrite the algorithm, it finds the best solution in a matter of seconds.

At this time, you are willing to take the algorithm as a black box and plug it in without too many questions. Still, this decision is crucial to your survival… it seems a situation where you could use some deep knowledge of how an algorithm works.

For our initial example, it turns out that the best possible combination we can find is rice, wheat flour and beans, for a total of **3,256,000 calories**. That would be quite a good result, compared to our first attempt, right?

You probably had already guessed the best combination, but if it seems too easy in the initial example, where we only have seven items, try the same with hundreds of different